The Sum of Coins

Jason Reeves (June 2025)

ABSTRACT. The classic "coin sum" problem was popularized in modern times by mathematician George Polya in an article published in 1956.[1] It is part of the branch of mathematics known as combinatorics, which is the mathematics of counting things in real and virtual worlds, and also has some overlaps with things like probability. Computer scientists especially love combinatoric problems, because these sorts of problems help them to demonstrate how very clever they are. The machines these scientists deal with are well suited, if appropriately prodded, to produce fast and accurate solutions to such problems which are performant at scale.

The inspiration for this write up was Project Euler Problem #31,[2] which uses coins from the United Kingdom. Given these coins, the pitiful human solving the problem must conjure up the number of ways these coins can total up to £2, or 200¢.

[1] George Polya. "On picture-writing". In: *American Math Monthly* 63.10 (1956).

[2] Project Euler. *Problem 31: Coin sums*.

L33+cod3 Rant

A cursory search of the Interwebs found a number of solutions to the coin sum problem having various $O()$ values, such as the recursive solution, the dynamic programming solution, and the space-optimized dynamic programming solution. So, finding a solution that could actually solve the problem in near-constant time was not a big problem.

It is said that there are companies out there that, when interviewing a prospective computer scientist, will actually expect them to produce a solution to such a problem in real time while watched by company agents. Perhaps I am not a very good computer scientist, but I can say that I never encountered this particular problem in my formal education, and even though I have a Bachelor's degree in both Computer Science and Mathematics, it took me a good few weeks of research and head scratching in order to even partially wrap my brain around it.

What I found difficult to comprehend, in addition to how the common solutions worked, was why companies would waste time requiring interviewees to jump through these hoops in order to secure a job in software engineering. Are these the sorts of problems the engineer is going to be required to solve day to day? Are you asking them to memorize something they could easily look up, wasting brain resources? How can you tell the difference between someone *actually* working through the problem live vs. regurgitating a memorized solution? The answers to these questions, I suspect, are "No", "No", and "We can't". What's the purpose of forcing someone to solve a problem *while you watch* that neither you nor they understand? Power dynamic, much?

I immediately apprehended *that* the software solutions I found online worked to find the answer, but I could not explain *how* they worked. My suspicion is that most people asking these questions in interviews, as well as most people being interviewed, are also ill-equipped to make such explanations. I make this conjecture for two reasons: 1) I have been keenly observing human nature for somewhere in the neighborhood of half a century, and 2) the explanations I found online were largely unsatisfying, full of unproductive hand-waving, or, even worse, statements such as "just memorize these formulas and you'll be fine".[3]

[3] *N.B.* – I wrote this article in mid-2025, but am editing it in late 2026 to prepare it for publication on my blog. As of right now, I've been using AI agents to write code exclusively for nearly a year. Most programmers are in the same boat, making such interview gymnastics seem even more silly.

Generating Functions

I will not make the attempt here to wax eloquent on generating functions. Suffice it to say 1) they are weird, and 2) they aren't real functions in the sense that you are ever going to plug anything into x and compute the result. Generating functions are a mathematical trick which probably could have gotten you burned at the stake in some New England towns I could mention at certain points in history. They are very powerful, and leverage the mystery of mathematics to make sense out of just about any series of numbers, or combinations of series of numbers. Folks like Knuth,[4] Polya, Harris,[5] and others have done a much better job than I could ever do of explaining these critters, and they seem to actually enjoy it quite a bit. Consider the following polynomial:

$$G(x) = a_0 1 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

Or, if you prefer a summation:

$$G(x) = \sum_{i=0}^{\infty} a_n x^n$$

Now, believe it or not, we can *encode* any sequence we like into this generating function by setting the sequence equal to $a_0 ... a_n$, and then the exponents of each term will tell us the position of the coefficient in the sequence. You don't need to necessarily know any of the mathematical properties underlying a particular sequence, but you can still use generating functions to determine properties of the sequence, extrapolate sequence elements you don't yet know, or combine sequences to determine properties of their combinations. That's all fun and junk, but we're not exactly going to do that today.

---

[4] Robert M. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete mathematics*. Addison-Wesley, 1994.

[5] John M. Harris, Jeffry L. Hirst, and Michael J. Mossinghoff. *Combinatorics and graph theory*. Springer New York, 2008.

Polya's Problem Statement

Given typical American currency options (pennies, nickels, dimes, quarters, and half-dollars), and a putatively infinite supply of each, how many ways are there to arrive at an amount of 50¢ using these coins? What about $1? Like many problems in this class, it *sounds* simple, until you start putting pencil to paper to actually solve it.

Polya's Construction

George Polya, in the aforementioned article published in 1956 titled "On Picture-Writing", set the coin sum problem up to be solved with generating functions where the coefficients of the function correspond to the ways to arrive at a certain value n, where n is the exponent of each term x^n.

Cents

In Polya's formulation, which I'm modifying *ever so slightly* so that I can make it make sense in my brain, he started with this base case:

$$C_i(x) = 1 + x + x^2 + \ldots + x^i$$

Don't get too excited. I'm using i in the equation above to represent an *item* in a series, not $\sqrt{-1}$. I *was* using n but I thought people might get confused later when I used $N_n(x)$ to refer to the generating function for nickels, so I changed it. In this generating function, the C represents *cents*. This function is a representation of the different ways we can arrive at a given i in value using only pennies. Since there is one and only one way to use pennies to arrive at a certain value[6] it makes sense that all the coefficients of this infinite polynomial are, in fact, 1.

Now we're starting to have fun, perhaps. Perhaps not. As my mom used to say, "You can't help some people." If you remember studying infinite series, you'll remember[7] that the infinite series represented by $C_i(x)$ has a *closed form*, namely $\frac{1}{1-x}$. This will become a recurring[8] and a very useful theme.

Nickels

Next up is the generating function which will encode how to arrive at a given value i using any number of nickels. Now it should be evident to every mother's son of you that you cannot construct a value of, say, '7' using only nickels. If you truly believe otherwise, perhaps you should consider running for US Congress. Without further ado, here is the generating function for values that can be formed from any number of nickels:

$$N_i(x) = 1 + x^5 + x^{10} + \ldots + x^i = \frac{1}{1-x^5}$$

I have helpfully provided the closed form for this infinite series as well.

Dimes, Etc.

If you guessed that dimes are next, good for you. If you guessed that something else was next, you are still a quality individual with dynamic proclivities. Here are the generating functions for dimes, quarters, and half-dollars, all at once, along with their closed forms:

$$D_i(x) = 1 + x^{10} + x^{20} + \ldots + x^i = \frac{1}{1-x^{10}}$$

$$Q_i(x) = 1 + x^{25} + x^{50} + \ldots + x^i = \frac{1}{1-x^{25}}$$

$$H_i(x) = 1 + x^{50} + x^{100} + \ldots + x^i = \frac{1}{1-x^{50}}$$

Nitty

A word about the *missing* terms in these infinite series. For example, why doesn't $H_i(x)$ have a x^{20} term? Because there's no way[9] to arrive at a value of '20' using only half dollars. So, *technically* those terms are there, but there coefficients are all 0, so we omit them for brevity.

It is a truism among those who regularly jockey with generating functions that if you want to establish an AND relationship among 2 or more generating functions, then you must multiply the sequences together. Better said, multiplication of generating functions encodes combining the possibilities represented by the constituent functions. It also suffices to simply multiply the closed forms of such sequences. Doing so allows you to analyze the combined sequences, which can be very healthy and profitable.

The resulting generating function representing combining the possibilities of all forms of currency aforementioned, therefore, is:

$$A_i(x) = \frac{1}{(1-x)(1-x^5)(1-x^{10})(1-x^{25})(1-x^{50})}$$

In this case, the A in $A_i(x)$ stands for *All*.

Gritty

Now, don't get all silly and try to multiply this thing out. I mean, you *can* if you're into that kind of thing, but in this case, happily there is no need. Another way to state what we're trying to get out of $A_i(x)$ is:

$$A_i(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + ... + a_ix^i$$

In the second form, we can see, based on our previous discussion, that the series a_i is used as the coefficients of this polynomial. In $N_i(x)$, the coefficients n_i represent the number of ways you can use a nickel to arrive at the value i, where i is the exponent of the coefficient's x^i term. In the second form of $A_i(x)$, then, the series A_i represents the number of ways to arrive at the value i, where i is the exponent of A_i's corresponding x^i term, but in this case we're pulling from the pool of coins consisting of all denominations, because we multiplied all the series together. Neat, huh? Now, in order to find out how many ways we can reach 50¢, we *only* need to find the coefficient of the x^{50} term in $A_i(x)$. Ain't that just like a combinatorial problem? You see the light at the end of the tunnel, and it turns out to be an oncoming train.

Reductio Ad Recursio

Polya uses his faculty of *mathematical intuition* to take a wild guess that we can suss out the coefficients in $A_i(x)$ by taking it apart again to gain some insight as to how it can be put back together.

He says, "Assume, for example, that we have already obtained the expansion of the product of the first two factors," by which he means the expansion of the multiplication of the closed forms of $C_i(x)$ and $N_i(x)$, which in our slightly modified formulation would look like:

$$\frac{1}{(1-x)(1-x^5)} = n_0 + n_1x + n_2x^2 + \ldots \qquad (\textit{equation N})$$

Polya continues "... and we wish to go on hence to three factors," by which he means the expansion of the multiplication of the closed forms of $C_i(x)$, $N_i(x)$, and $D_i(x)$, which would be:

$$\frac{1}{(1-x)(1-x^5)(1-x^{10})} = d_0 + d_1x + d_2x^2 + \ldots \qquad (\textit{equation D})$$

Polya concludes by deriving an ingenious recursive formula for the coefficients. Given (*equation N*) and (*equation B*), it follows that:

$$(d_0 + d_1x + d_2x^2 + \ldots)(1 - x^{10}) = n_0 + n_1x + n_2x^2 + \ldots$$

His next step which he simply whips out with a "we find that ..." required me to employ pencil and paper to properly understand. Enlightenment comes from observing what these coefficients *do*:

$$\begin{array}{cccccccccccccccc} & d_0 & + & d_1x & + & d_2x^2 & + & \ldots & + & d_{10}x^{10} & + & d_{11}x^{11} & + & \ldots \\ - & \ldots & \ldots & \ldots & \ldots & \ldots & \ldots & \ldots & + & d_0x^{10} & + & d_1x^{11} & + & \ldots \\ \hline \end{array}$$

We can see that the result of this subtraction should be equal to the series $n_0 + n_1x + n_2x^2 + \ldots$. Rest easy, though. We're not going to do actual arithmetic on these two infinite polynomials, but we can reason about what would happen if we did, and this without having to do every tedious step. If we *were* going to subtract these polynomials, set them equal to (*equation N*), and try to figure out what these coefficients are, the first thing we would do is organize like powers of x so that we could add those coefficients up. When we do that, we notice that:

$$x^{10} \text{ coefficients: } d_{10} - d_0 = n_{10} => d_{10} = n_{10} + d_0$$
$$x^{11} \text{ coefficients: } d_{11} - d_1 = n_{11} => d_{11} = n_{11} + d_1$$
$$x^{12} \text{ coefficients: } d_{12} - d_2 = n_{12} => d_{12} = n_{12} + d_2$$

Therefore, we may conclude, as Polya does, that $d_i = n_i + d_{i-10}$, which it most certainly does.

A Little Pedantry

I'll connect some dots for you so that you can easily see the connection back to the base case, and then we'll take a wild leap of faith that you can justify all by yourself with a pencil, a piece of paper, and a little furrowing of the brow.

On with the pedantry. We'll do the same operation we just did, but we'll derive the coefficients of the *product* of $N_i(x)$ and $C_i(x)$ in terms of coefficients of $N_i(x)$ and $C_i(x)$ individually. To wit:

$$\frac{1}{(1-x)} = c_0 + c_1x + c_2x^2 + \ldots \qquad (\textit{equation C})$$

$$\frac{1}{(1-x)(1-x^5)} = n_0 + n_1x + n_2x^2 + \ldots \qquad (\textit{equation D})$$

Then, we observe:

$$\begin{array}{cccccccccccccccc} & n_0 & + & n_1x & + & n_2x^2 & + & \ldots & + & n_5x^5 & + & n_6x^6 & + & \ldots \\ - & \ldots & \ldots & \ldots & \ldots & \ldots & \ldots & \ldots & + & n_0x^5 & + & n_1x^6 & + & \ldots \\ \hline \end{array}$$

And of course, the result of this subtraction must be equal to $c_0 + c_1x + c_2x^2 + \ldots$. Once again grouping together the various exponents of x, we find:

$$x^5 \text{ coefficients: } n_5 - n_0 = c_5 => n_5 = c_5 + n_0$$
$$x^6 \text{ coefficients: } n_6 - d_1 = c_6 => n_6 = c_6 + n_1$$
$$x^7 \text{ coefficients: } n_7 - n_2 = c_7 => n_7 = c_7 + n_2$$
$$\therefore n_i = c_i + n_{i-5}$$

The Leap

We are now going to leap, but let the record reflect that before we did so, we absolutely, positively, and without a doubt actually *looked* prior to said leap. Some of you with advanced math degrees who do this kind of thing every day[10] are probably wondering what took me so long to get here, because you had already either solved the problem or tuned out completely after my first paragraph, way, way up above. Oh well. This paper is probably NOT for you, but I'm happy for you to read it, and will kindly take any constructive criticism you have to offer. For myself, an amateur intellectual, I have to look a lot closer before I make leaps like this, and for a lot longer. Having been around for some time, things don't heal like they used to, and so when I make a leap, I have to make absolutely sure it's *safe* and that I will have a soft place to land.

Having assured myself that this leap will not be fatal,[11] I am going to take a deep breath and make the following leap:

$$a_i = \textit{previous coin added}_i + \textit{current coin}_{i-\textit{value of current coin}} \qquad (\textit{the leap})$$

What are we trying to say here? I'll get to that. In the meantime, refer to $A_i(x)$ defined in the *Nitty* section above. Now, suppose that we *build up* to $A_i(x)$ gradually, instead of all at once:

$$Z_i(x) = C_i(x)$$
$$J_i(x) = Z_i(x) \times N_i(x)$$
$$K_i(x) = J_i(x) \times D_i(x)$$
$$L_i(x) = K_i(x) \times Q_i(x)$$
$$A_i(x) = L_i(x) \times H_i(x)$$

So now, $Z_i(x)$ is identical to $C_i(x)$, encoding the number of ways we can use *cents*, or pennies, to create a given value. Likewise, $J_i(x)$ encodes the number of ways we can use an infinite number of coins including *cents AND nickels* to create a given value. $K_i(x)$ adds *dimes* to the mix, and we keep building until we get to $A_i(x)$, which is the same $A_i(x)$ we've seen before,[12] encoding the number of ways to get to a certain value using *all* the coins, if only we could find the coefficients of a given power of x, which we[13] can now do.

Let us try to be *slightly* more mathematically rigorous. Let's define a ordered sequence *COINS* with elements $(1, 5, 10, 25, 50)$. Let's also define a function *element_at()* which operates on *COINS* and an integer to yield the element of *COINS* at that index, and another function *size()* which will give us the size of the sequence. Then:

$$A_i(x) = \prod_{v=1}^{\textit{size(COINS)}} \frac{1}{1 - x^{\textit{element\_at(COINS,v)}}}$$

If we take a_i above to represent the coefficients of $A_i(x)$, etc., then:

$$a_i = q_i + h_{i-50}$$
$$q_i = d_i + q_{i-25}$$
$$d_i = n_i + d_{i-10}$$
$$n_i = c_i + n_{i-5}$$
$$c_i = 1$$

... going back up the stack reveals other insights ...

$$n_i = 1 + n_{i-5}$$
$$d_i = 1 + n_{i-5} + d_{i-10}$$
$$q_i = 1 + n_{i-5} + d_{i-10} + q_{i-25}$$
$$a_i = 1 + n_{i-5} + d_{i-10} + q_{i-25} + h_{i-50}$$

So, then, the number of ways we can create a value, of, say '100' with only cents is 1. It doesn't matter what the value is, there is only one number of ways to do it. *Quod erat demonstrandum*.

The number of ways we can create a given value using cents AND nickels is to take the number of ways you can create that value with cents, and add it to the number of ways that you can create the value *minus 5*[14] with cents AND nickels.

The number of ways we can create a given value using cents AND nickels AND dimes is to take the number of ways you can create that value with cents, and add it to the number of ways that you can create the value *minus 5*[15] with cents AND nickels, and add THAT to the number of ways that you can create the value *minus 10* using cents AND nickels AND dimes.

As you can see, this is getting quite complicated. I don't think we technically *need* a computer quite yet to figure it out, but since spreadsheets are the second best way to do anything, we might consider employing one. Or, in order to get the worst of both worlds, we'll just implement a table in LaTeX.

The Tables Are Turning

Let's walk through the construction of a table in order to help us understand how these coefficients are interrelated.

values coins	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50
{}	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

I snuck an edge case in there on ya! If there are no coins, i.e. if the set of available coins is equal to the null set, there is 1 way to choose 0,[16] but there are 0 ways to choose coins from the null set to get to any other value.

values coins	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50
{}	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
{1}	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
{1,5}	1	1	1	1	1	2																																													

It should already make sense, since we've discussed it *ad nauseam* above, that there is one way to make each value using nothing but pennies. That reality is reflected in row $\{1\}$.

Before we go further, let's walk through the first few values of $\{1,5\}$. It should be evident that you can't use a nickel to make any amount less than 5¢, so since we can only use cents here, there is one way to create a value of 1¢, 2¢, 3¢, or 4¢. BUT, once we get to column 5 (representing, of course, the *value* of 5) we see there are two ways. That is because we can *either* choose 5 one-cent coins, or 1 five-cent coin in order to achieve that value. There are no other options. Reflect back on our recurrence relation $n_i = c_i + n_{i-5}$. Let's set $i = 5$, and we get $n_5 = c_5 + n_0$, which becomes $n_5 = 1 + 1 = 2$. Yes, Virginia, $1 + 1$ still equals 2, whether or not there's a Santa Claus.

If you prefer, you can think of a cell value being formed by adding the value immediately *above* with the value i places to the *left* in the current row. The cells in play are highlighted in the table above. It should make sense to you that our recurrence can't have *negative* subscripts, since doing so would cause our cheese to slide off our cracker, which would be, at the very least, lamentable. So, then, more rigor[17]:

$$a_i = \begin{cases} q_i + h_{i-50} & i \geq 50 \\ q_i & i < 50 \end{cases}$$

$$\mathrm{q}_i = \begin{cases} d_i + q_{i-25} & i \geq 25 \\ d_i & i < 25 \end{cases}$$

$$\mathrm{d}_i = \begin{cases} n_i + d_{i-10} & i \geq 10 \\ n_i & i < 10 \end{cases}$$

$$\mathrm{n}_i = \begin{cases} c_i + n_{i-5} & i \geq 5 \\ c_i & i < 5 \end{cases}$$

$$\mathrm{c}_i = \begin{cases} 1 & i \geq 0 \end{cases}$$

The c_i equation really didn't *need* the bracket treatment, but it would have felt left out otherwise, and it doesn't cost me any extra to dress it up now and again.

Let's finish up row $\{1,5\}$ and add the next row:

values coins	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50
{}	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
{1}	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
{1,5}	1	1	1	1	1	2	2	2	2	2	3	3	3	3	3	4	4	4	4	4	5	5	5	5	5	6	6	6	6	6	7	7	7	7	7	8	8	8	8	8	9	9	9	9	9	10	10	10	10	10	11
{1,5,10}	1	1	1	1	2	2	2	2	2	2	4	4	4	4	4	6	6	6	6	6	9	9	9	9	9	12	12	12	12	12	16	16	16	16	16	20	20	20	20	20	25	25	25	25	25	30	30	30	30	30	36

Notice how the highlighted cells in $\{1,5,10\}$ are merely *pulled down*, as it were, from the row above? This is due to the fact that we can't have negative subscripts, and there's no way to make anything less than 10¢ using a dime.

I will now quit dilly-dallying and give you the whole table:

values coins	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50
{}	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
{1}	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
{1,5}	1	1	1	1	1	2	2	2	2	2	3	3	3	3	3	4	4	4	4	4	5	5	5	5	5	6	6	6	6	6	7	7	7	7	7	8	8	8	8	8	9	9	9	9	9	10	10	10	10	10	11
{1,5,10}	1	1	1	1	1	2	2	2	2	2	4	4	4	4	4	6	6	6	6	6	9	9	9	9	9	12	12	12	12	12	16	16	16	16	16	20	20	20	20	20	25	25	25	25	25	30	30	30	30	30	36
{1,5,10,25}	1	1	1	1	1	2	2	2	2	2	4	4	4	4	4	6	6	6	6	6	9	9	9	9	9	13	13	13	13	13	18	18	18	18	18	24	24	24	24	24	31	31	31	31	31	39	39	39	39	39	49
{1,5,10,25,50}	1	1	1	1	1	2	2	2	2	2	4	4	4	4	4	6	6	6	6	6	9	9	9	9	9	13	13	13	13	13	18	18	18	18	18	24	24	24	24	24	31	31	31	31	31	39	39	39	39	39	50

I *do* apologize for showing the entire table like that, and I can assure you it is not from a lack of modesty that I do so. Neither Polya nor Knuth showed the whole table, and the holes in the table resulted in quizzical looks and uncertain sentiment from yours truly. I am attempting to be *very explicit* because this problem really stumped me for quite a while, and I'm betting it has stumped others who saw things they didn't think were obvious, but kept reading things like *obviously, it follows that ...* which didn't get them any closer to comprehension. At least I like to think that there are those out there as thick as me. Maybe I'm just imagining those people are out there just to make myself feel better. Oh, bother.

---

[6] Keep in mind that order doesn't matter, a penny is a penny.

[7] Maybe?

[8] Heh-heh.

[9] As joshed about previously...

[10] ... and twice before breakfast

[11] If *you* remain unassured, *you* may assuage yourself with paper and pencil.

[12] AS PER MY PREVIOUS EMAIL.

[13] I believe!

[14] The value of the nickel.

[15] Once again, the value of the nickel.

[16] i.e. – standing pat.

[17] But again, not *much* more...

Software Solutions

If you manually worked out that entire table in the previous section by hand, kudos to you. I did not. I used a modern machine known as a *computer*, a wonderful machine which has inspired the following verses:

> At work they bought a new machine.
> I wish that they would sell it.
> It doesn't do the things I want,
> But only what I tell it.

If you thought *I* was pedantic, you may not have had much experience with your friendly neighborhood computer. Computers are the absolute worst. With a toddler, you can say "come here" and receive some fairly sensible responses on average. With a computer, you have to describe to them what "legs" are and what "walking" is and exactly how to do it. As much flack as toddlers get, they are, again, *on average*, much smarter than an unprogrammed computer. An unprogrammed computer will just sit there no matter how loudly you yell. It does not even care about candy very much, which is a cheap way to get a toddler to do pretty much anything that is within their power to do. If you are very lucky, it *might*, unprompted, blink lights at you nonchalantly.

So, in order for a computer to do useful work for us, we have to break things down into teeny tiny bits, and understand that the computer has no inherent intelligence, not to mention horse sense. It is really much better at mathematics than a toddler, being able to answer questions like $182832832 + 923949234$ with the greatest of ease, but it can't even display that answer to the screen or print it out without a *lot* of additional instructions. A toddler has *motivations* and the capability to *reason* which makes them able to figure out many problems a computer could never do. Toddlers are also cute, and computers are decidedly not. That said, once you come up with the correct series of steps a computer has to do in order to solve a problem, it can solve it reliably and consistently - it will not deviate from its programming whatsoever.[18] This is also an advantage over toddlers. Similar, but distinct from, the toddler's mantra of "If it's fun once, it's fun 1000 times," a computer never gets tired of doing the same thing, over and over again. Humans get bored. Computers do not.

Enough about toddlers vs. computers. I'm sure you get the point I'm trying to make. At least I hope you do. Let's talk about the three most popular ways to solve the coin sum problem: Recursion, Memoization, and Space Optimized Memoization.

Recursion

Recursion has the advantage that it is *really cool*. Here is some Ruby code that demonstrates how you can solve the coin sum problem with recursion:

```ruby
def coin_sum(coins, target, i = 0)
  return 1 if target == 0
  return 0 if target < 0 || i == coins.length

  # Include the coin at index and stay on the same index
  with_coin = coin_sum(coins, target - coins[i], index)

  # Exclude the coin at index and move to the next index
  without_coin = coin_sum(coins, target, i + 1)

  with_coin + without_coin
end

# Example usage
coins = [1, 2, 5]
target = 5
puts coin_sum(coins, target)  # Output: 4
```

Clever, huh? Unfortunately, as the target amount T grows, the computer's computation slows. This solution has $O(2^T)$ time complexity, and $O(T)$ space complexity. The recursion tree is growing exponentially based on the target amount and the size of the set of coins. So for large targets, this solution doesn't scale well.

What is going on in this code is that the recursion at each step branches off in two directions 1) use the new coin added in this loop, or 2) don't use it. The reason this solution is not as effective is that there are certain calculations happening more than once. A memoized solution should help.

Memoization

Here is a *memoized* solution in Ruby:

```ruby
1  def coin_sum(coins, target)
2    n = coins.length
3    dp = Array.new(n + 1) { Array.new(target + 1, 0) }
4
5    # Base case: there is 1 way to make sum 0 (use no coins)
6    (0..n).each { |i| dp[i][0] = 1 }
7
8    # Fill the dp table
9    (1..n).each do |i|
10     (1..target).each do |j|
11       if j < coins[i - 1]
12         dp[i][j] = dp[i - 1][j]
13       else
14         dp[i][j] = dp[i - 1][j] + dp[i][j - coins[i - 1]]
15       end
16     end
17   end
18
19   dp[n][target]
20 end
21
22 # Example usage
23 coins = [1, 2, 5]
24 target = 5
25 puts coin_sum(coins, target)  # Output: 4
```

Memoization, also known as *dynamic programming*,[19] is descriptive of methodologies which remember previous calculations so that they don't have to be done again. This solution has $O(nT)$ complexity in space and time, where n is the number of coins and T is the target amount.

At the heart of this solution are lines 9-17 which fill in the *dp* table:

```ruby
9    (1..n).each do |i|
10     (1..target).each do |j|
11       if j < coins[i - 1]
12         dp[i][j] = dp[i - 1][j]
13       else
14         dp[i][j] = dp[i - 1][j] + dp[i][j - coins[i - 1]]
15       end
16     end
17   end
```

This looks *suspiciously* similar to our "not *much* more" rigorous recurrence relations laid out in *The Tables Are Turning* above. I'll reproduce the definition of a_i here:

$$a_i = \begin{cases} q_i + h_{i-50} & i \geq 50 \\ q_i & i < 50 \end{cases}$$

Allow me to attemptify linguisticating this in Americanese for you. In the final loop of the *each* on line 9, assuming we had the list of coins $\{1, 5, 10, 25, 50\}$ stored in *coins* and our *target* is set to 50, we're going to be building the last row in the full table at the end of *The Tables Are Turning*. The last coin added would be the half dollar, so on line 11 if $j < coins[i-1]$ will be $j < 50$, then we effectively *pull down* the value from the same column one row above via $dp[i][j] = dp[i-1][j]$, which is exactly what we're doing in the second case of the a_i recurrence relation. Contrariwise, in the *else* block on line 14, we see the first case represented as $dp[i][j] = dp[i-1][j] + dp[i][j - coins[i-1]]$. This is the same thing as $a_i = q_i + h_{i-50}$ when $i \geq 50$. I trust you can satisfy yourself that the same thing is happening in the recursive version of the code above, albeit a bit more inefficiently.

Space-optimized Memoization

Of course, we're going to be throwing away most of the two-dimensional *dp* array in the non-space-optimized solution above, so we can do better on space utilization if we throw those results away and use more of an accumulator strategy:

```ruby
def coin_sum(coins, target)
  n = coins.length
  dp = Array.new(target + 1, 0)
  dp[0] = 1  # Base case: one way to make sum 0

  (1..n).each do |i|
    (coin..target).each do |j|
      dp[j] += dp[j - coins[i]]
    end
  end

  dp[target]
end

# Example usage
coins = [1, 2, 5]
target = 5
puts coin_sum(coins, target)  # Output: 4
```

This solution still has $O(nT)$ time complexity, but we improve the space complexity to $O(T)$. Algorithmically, we're still doing the same thing as in section 6.2, but we've tweaked the structure slightly to optimize the usage of space.

Enlightenment

I have seen some doozies while researching this problem with regard to how people explain what exactly is going on. I was regularly dissatisfied with the lack of explanation with regard to *why* each iteration of the loops or the recursion is actually tabulating the *ways* that a coin sum can be arrived at with the given coin options at a given step. I grant this might have been blisteringly obvious for YOU, dear reader, but it was not for your humble correspondent.

This problem is squarely in the field of combinatorics, although it is not a simple $\binom{n}{k}$ combination or permutation problem. This problem is in the arena of *partition* problems[20] but it does not fit neatly under that umbrella either. Unfortunately (or fortunately, if you kind of like solving hard puzzles) there are no pat answers to questions about why the code examples above work. There *are* answers, to be sure, and I think we've explored some of them in this paper. But they are not *simple* answers and they require some time and space to explain and understand. Life is like that.

I say all this to say that this is a *hard* problem. But, with clever mathematical simplifications, we can master the entire solution space without involving ourselves in a bunch of tedium simply by asking a computer (nicely!) to do the monotonous addition and tabulation for us.

Ad Fontes

In addition to the resources cited above, I also scoured many websites and YouTube videos too numerous to mention, and, alas, I did not keep track of them, so I could not mention them even if I wanted to.[21] For the code samples, I used ChatGPT and modified the output for the sake of consistency. In order to get a full-orbed understanding of this problem and its history, I highly recommend checking out the original source material.

---

[18] That is, as long as *deterministic* programming is what's being discussed. AI is *non*-deterministic, meaning you *don't* get the same answer every time. Believe it or not, sometimes this is good. Let the reader understand.

[19] Whence *dp* in the code ...

[20] Another breed of deceptively simple problems which are actually hard.

[21] I do not.